ROTEIRO DE ESTUDO

Revisão de MySQL

Banco de dados de uma loja on-line

Conexão e usuários; criação de banco e tabelas; chaves primárias e estrangeiras; inserção de dados; filtros; ordenação; agrupamentos; funções de agregação; junções; subconsultas; views; índices e análise de consultas.

Prof. Márcio Fantini

Colégio Técnico da Universidade Federal de Minas Gerais

COLTEC/UFMG

Material de revisão com exemplos, atividades e gabarito comentado

MySQL 8.0 ou superior

Sumário

1 Apresentação **2**

2 Conhecendo os dados antes dos comandos SQL **2**

2.1 Tabela de clientes . . . 2

2.2 Tabela de produtos . . . 3

2.3 Tabela de vendedores . . . 3

2.4 Tabela de vendas . . . 3

3 Atividades iniciais de leitura das tabelas **4**

4 Entrando no MySQL e criando um usuário **7**

4.1 Abrindo o shell como administrador . . . 7

4.2 Criando o usuário da aula . . . 7

5 Modelo do banco de dados **8**

6 Criação do banco e das tabelas **9**

6.1 Tarefa . . . 9

7 Inserção dos dados **11**

8 Revisão dos comandos fundamentais **13**

8.1 SELECT, aliases e expressões calculadas . . . 13

8.2 Filtros com WHERE . . . 13

8.3 Ordenação e limitação . . . 14

8.4 Funções de agregação e agrupamentos . . . 14

8.5 UPDATE e DELETE . . . 15

9 Exercício 1: construção do banco **15**

10 Exercício 2: consultas básicas e intermediárias **15**

11 Revisão de JOIN **16**

12 Exercício 3: consultas com JOIN **17**

13 Gabarito comentado **17**

13.1 Consultas básicas e intermediárias . . . 17

13.2 Consultas com JOIN . . . 19

14 Perguntas para discussão **20**

15 Checklist de aprendizagem **21**

1 Apresentação

Este roteiro propõe uma revisão prática dos principais conteúdos de MySQL. O aluno construirá um pequeno banco de dados para uma empresa de vendas on-line e, depois, resolverá consultas de dificuldade crescente.

Objetivos de aprendizagem

Ao concluir o roteiro, o estudante deverá ser capaz de:

- entrar no cliente de linha de comando do MySQL;
- criar um usuário e conceder privilégios de maneira controlada;
- criar bancos, tabelas, chaves primárias e chaves estrangeiras;
- inserir, consultar, alterar e excluir registros;
- empregar `WHERE`, `ORDER BY`, `GROUP BY` e `HAVING`;
- aplicar `COUNT`, `MAX`, `MIN`, `SUM` e `AVG`;
- combinar tabelas com `INNER JOIN` e `LEFT JOIN`;

Metodologia sugerida

O roteiro começa com uma etapa obrigatória sem computador: o estudante deverá escrever à mão os comandos que recorda. Somente depois dessa tentativa inicial ele deverá consultar as explicações, executar os exemplos no MySQL, observar os resultados e comparar sua solução com o gabarito.

2 Conhecendo os dados antes dos comandos SQL

Antes de criar o banco e escrever comandos, observe atentamente as quatro tabelas que serão utilizadas. Essa leitura inicial ajuda a identificar entidades, atributos, chaves, relacionamentos e perguntas que poderão ser respondidas pelo banco.

2.1 Tabela de clientes

Tabela 1: Dados da tabela `clientes`.

ID	Nome	Cidade	UF	E-mail	Cadastro
1	Ana Souza	Belo Horizonte	MG	ana@email.com	10/02/2026
2	Bruno Lima	Contagem	MG	bruno@email.com	15/03/2026
3	Carla Mendes	São Paulo	SP	carla@email.com	20/04/2026
4	Diego Alves	Curitiba	PR	diego@email.com	05/05/2026
5	Elisa Rocha	Belo Horizonte	MG	elisa@email.com	12/06/2026

Cada linha representa um cliente. O campo `cliente_id` identifica a linha, enquanto os demais campos descrevem o cliente.

2.2 Tabela de produtos

Tabela 2: Dados da tabela `produtos`.

ID	Nome	Categoria	Preço (R$)	Estoque	Ativo
1	Notebook Pro 15	Informática	4.500,00	12	Sim
2	Mouse sem fio	Informática	120,00	50	Sim
3	Cadeira Office	Móveis	980,00	8	Sim
4	Monitor 27	Informática	1.600,00	15	Sim
5	Luminária LED	Casa	180,00	30	Sim

2.3 Tabela de vendedores

Tabela 3: Dados da tabela `vendedores`.

ID	Nome	Setor	Comissão (%)	Admissão
1	Felipe Costa	Corporativo	5,00	10/03/2022
2	Gabriela Reis	Varejo	4,50	18/07/2023
3	Henrique Melo	Varejo	4,00	22/01/2024
4	Isabela Nunes	Corporativo	5,50	01/09/2021
5	João Martins	On-line	3,50	14/02/2025

2.4 Tabela de vendas

Tabela 4: Dados da tabela `vendas`.

Venda	Cliente	Produto	Vend.	Data	Qtd.	Preço unit.	Status
1	1	1	1	01/08/2026	1	R$ 4.400,00	CONCLUÍDA
2	2	2	2	03/08/2026	3	R$ 115,00	CONCLUÍDA
3	1	4	5	07/08/2026	2	R$ 1.550,00	CONCLUÍDA
4	3	3	4	10/08/2026	1	R$ 950,00	PENDENTE
5	5	5	2	15/08/2026	4	R$ 175,00	CANCELADA

Na tabela de vendas, os campos `cliente_id`, `produto_id` e `vendedor_id` armazenam identificadores encontrados nas outras tabelas. Assim, para descobrir os nomes envolvidos em uma venda, é necessário relacionar as tabelas.

Atenção

O preço mostrado na tabela de vendas é o preço efetivamente praticado naquela operação. Ele pode ser diferente do preço atual encontrado no cadastro de produtos, por causa de descontos ou alterações posteriores no catálogo.

3 Atividades iniciais de leitura das tabelas

Resolva as questões a seguir apenas observando as Tabelas 1 a 4. Neste momento, ainda não é necessário escrever comandos SQL. Registre o raciocínio e a resposta em um arquivo texto, usando o seu editor de texto preferido.

Atividade 1 — reconhecimento das entidades

Quais são as quatro entidades representadas no banco? O que representa uma linha de cada tabela?

Resposta

Atividade 2 — identificação das chaves

Qual coluna parece identificar unicamente cada registro nas tabelas? Na tabela `vendas`, quais colunas parecem fazer referência às outras tabelas?

Resposta

Atividade 3 — filtros simples

Quais clientes são de Minas Gerais? Escreva seus nomes e suas cidades.

Resposta

Atividade 4 — comparação e ordenação

Qual é o produto mais caro? Qual é o mais barato? Em que ordem os cinco produtos ficariam se fossem organizados do maior para o menor preço?

Resposta

Atividade 5 — agregação

Quantas unidades de produtos existem no estoque, considerando a soma dos cinco registros? Mostre o cálculo realizado.

Resposta

Atividade 6 — agrupamento

Quantos produtos existem em cada categoria? Qual categoria possui mais produtos?

Resposta

Atividade 7 — relacionamento entre tabelas

Na venda de número 3, identifique o cliente, o produto e o vendedor. Explique como os identificadores da tabela `vendas` permitiram encontrar esses nomes.

Resposta

Atividade 8 — coluna calculada

Calcule o valor total de cada venda multiplicando quantidade por preço unitário. Qual venda possui o maior valor?

Resposta

Atividade 9 — condição e soma

Qual é o faturamento total se forem consideradas somente as vendas concluídas? Não inclua a venda pendente nem a cancelada. Mostre o cálculo.

Resposta

Atividade 10 — ausência de correspondência

Quais clientes não aparecem em nenhuma venda? Quais vendedores não aparecem em nenhuma venda? Esse tipo de pergunta será importante quando estudarmos `LEFT JOIN`.

Resposta

Do raciocínio manual para o SQL

As atividades anteriores antecipam operações que serão automatizadas pelo MySQL: selecionar colunas, filtrar linhas, ordenar, contar, somar, calcular médias, agrupar e relacionar tabelas. A partir da próxima seção, transformaremos cada raciocínio em um comando executável.

4 Entrando no MySQL e criando um usuário

4.1 Abrindo o shell como administrador

Em uma instalação tradicional do MySQL, abra o terminal e execute:

Código 1: Entrada no MySQL como administrador

```
mysql -u root -p
```

O parâmetro `-u root` escolhe o usuário administrativo e `-p` solicita a senha. A senha não aparece enquanto é digitada. Em algumas instalações Ubuntu configuradas com autenticação pelo sistema operacional, pode ser necessário usar:

Código 2: Alternativa comum no Ubuntu

```
sudo mysql
```

Depois da conexão, confirme o servidor e a versão:

Código 3: Verificação da conexão

```
SELECT USER(), CURRENT_USER(), VERSION();
SHOW DATABASES;
```

Atenção

O usuário `root` deve ser reservado para tarefas administrativas. As atividades cotidianas devem ser realizadas com uma conta própria, com apenas os privilégios necessários. Não utilize senhas simples em ambientes reais.

4.2 Criando o usuário da aula

Crie o usuário `aluno_mysql`. Troque a senha de exemplo antes de usar o comando fora do laboratório:

Código 4: Criação do usuário

```
CREATE USER 'aluno_mysql'@'localhost'
IDENTIFIED BY 'Troque_Esta_Senha_2026!';
```

Há duas estratégias úteis para a concessão de privilégios.

Estratégia A — laboratório isolado: permite criar bancos e trabalhar com todos os objetos do servidor didático.

Código 5: Privilégios amplos em servidor exclusivamente didático

```
GRANT CREATE, ALTER, DROP, INDEX,
      SELECT, INSERT, UPDATE, DELETE,
      CREATE VIEW, SHOW VIEW
ON *.* TO 'aluno_mysql'@'localhost';
```

Estratégia B — recomendada: o administrador cria o banco e concede privilégios somente sobre ele.

Código 6: Privilégios restritos ao banco da atividade

```
CREATE DATABASE lojaVirtual
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

GRANT ALL PRIVILEGES
ON lojaVirtual.* TO 'aluno_mysql'@'localhost';

SHOW GRANTS FOR 'aluno_mysql'@'localhost';
```

Saia da sessão administrativa e conecte-se com o novo usuário:

Código 7: Nova conexão com a conta da aula

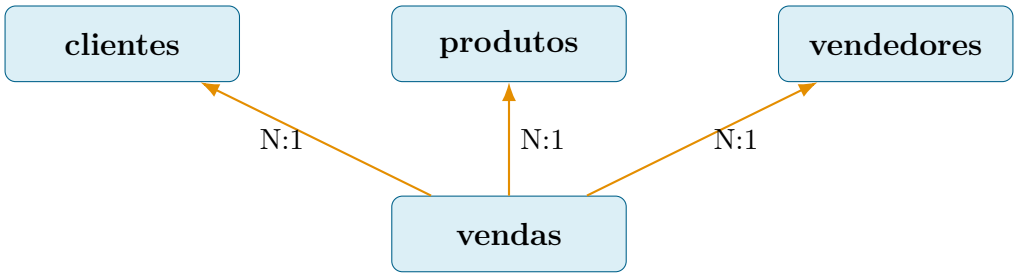
```
exit
mysql -u aluno_mysql -p
```

Atenção

Em versões atuais, `GRANT` não cria o usuário automaticamente. Primeiro use `CREATE USER`; depois conceda os privilégios. Normalmente não é necessário executar `FLUSH PRIVILEGES` após `CREATE USER` ou `GRANT`.

5 Modelo do banco de dados

O banco contém quatro entidades: clientes, vendedores, produtos e vendas. Cada registro de `vendas` representa a venda de um produto, em determinada quantidade, para um cliente e por intermédio de um vendedor.



Chaves

Uma **chave primária** identifica unicamente cada linha. Uma **chave estrangeira** referencia a chave primária de outra tabela e ajuda a garantir a integridade referencial. Por exemplo, todo `cliente_id` presente em `vendas` deve existir em `clientes`.

Atenção

Este modelo foi simplificado para a revisão. Em um sistema comercial, uma venda pode possuir vários produtos. Nesse caso, o modelo normalmente inclui as tabelas `vendas` e `itens_venda`, formando uma relação muitos-para-muitos entre vendas e produtos.

6 Criação do banco e das tabelas

6.1 Tarefa

Antes de ver a explicação a seguir (com as respostas) tente fazer por sua própria conta o que é pedido:

1. Crie o banco de dados lojaVirtual
2. Crie as quatro tabelas do banco.
3. Insira os dados.

Você pode usar a linha de comando do terminal do mysql ou criar um arquivo .sql com as instruções. Somente depois de tentar fazer, siga para as explicações e respostas dadas a seguir.

Se o administrador adotou a Estratégia A, o próprio aluno pode criar o banco:

Código 8: Criação e seleção do banco

```sql
CREATE DATABASE IF NOT EXISTS lojaVirtual
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE lojaVirtual;
SELECT DATABASE();
```

Agora crie as tabelas na ordem correta: primeiro as tabelas independentes e, por último, a tabela que contém as chaves estrangeiras.

Código 9: Criação das quatro tabelas

```sql
CREATE TABLE clientes (
    cliente_id INT UNSIGNED AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    cidade VARCHAR(80) NOT NULL,
    uf CHAR(2) NOT NULL,
    email VARCHAR(120) NOT NULL,
    data_cadastro DATE NOT NULL,
    CONSTRAINT pk_clientes PRIMARY KEY (cliente_id),
    CONSTRAINT uq_clientes_email UNIQUE (email)
) ENGINE = InnoDB;

CREATE TABLE produtos (
    produto_id INT UNSIGNED AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    preco DECIMAL(10,2) NOT NULL,
    estoque INT UNSIGNED NOT NULL DEFAULT 0,
    ativo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_produtos PRIMARY KEY (produto_id),
    CONSTRAINT ck_produtos_preco CHECK (preco >= 0)
) ENGINE = InnoDB;
```

```
22
23 CREATE TABLE vendedores (
24     vendedor_id INT UNSIGNED AUTO_INCREMENT,
25     nome VARCHAR(100) NOT NULL,
26     setor VARCHAR(50) NOT NULL,
27     comissao DECIMAL(5,2) NOT NULL DEFAULT 0,
28     data_admissao DATE NOT NULL,
29     CONSTRAINT pk_vendedores PRIMARY KEY (vendedor_id),
30     CONSTRAINT ck_vendedores_comissao
31       CHECK (comissao BETWEEN 0 AND 100)
32 ) ENGINE = InnoDB;
33
34 CREATE TABLE vendas (
35     venda_id INT UNSIGNED AUTO_INCREMENT,
36     cliente_id INT UNSIGNED NOT NULL,
37     produto_id INT UNSIGNED NOT NULL,
38     vendedor_id INT UNSIGNED NOT NULL,
39     data_venda DATE NOT NULL,
40     quantidade INT UNSIGNED NOT NULL,
41     preco_unitario DECIMAL(10,2) NOT NULL,
42     status_venda VARCHAR(20) NOT NULL DEFAULT 'PENDENTE',
43     CONSTRAINT pk_vendas PRIMARY KEY (venda_id),
44     CONSTRAINT ck_vendas_quantidade CHECK (quantidade > 0),
45     CONSTRAINT fk_vendas_clientes FOREIGN KEY (cliente_id)
46       REFERENCES clientes (cliente_id)
47       ON UPDATE CASCADE ON DELETE RESTRICT,
48     CONSTRAINT fk_vendas_produtos FOREIGN KEY (produto_id)
49       REFERENCES produtos (produto_id)
50       ON UPDATE CASCADE ON DELETE RESTRICT,
51     CONSTRAINT fk_vendas_vendedores FOREIGN KEY (vendedor_id)
52       REFERENCES vendedores (vendedor_id)
53       ON UPDATE CASCADE ON DELETE RESTRICT
54 ) ENGINE = InnoDB;
```

Confira a estrutura criada:

Código 10: Inspeção do banco

```
1 SHOW TABLES;
2 DESCRIBE clientes;
3 DESCRIBE produtos;
4 DESCRIBE vendedores;
5 DESCRIBE vendas;
6 SHOW CREATE TABLE vendas;
```

7 Inserção dos dados

Só veja as respostas depois de tentar fazer.

Cada tabela receberá cinco registros. Observe que os identificadores citados em `vendas` devem existir previamente nas três tabelas relacionadas.

Código 11: Inserção de cinco clientes

```
INSERT INTO clientes
    (nome, cidade, uf, email, data_cadastro)
VALUES
    ('Ana Souza', 'Belo Horizonte', 'MG',
     'ana@email.com', '2026-02-10'),
    ('Bruno Lima', 'Contagem', 'MG',
     'bruno@email.com', '2026-03-15'),
    ('Carla Mendes', 'São Paulo', 'SP',
     'carla@email.com', '2026-04-20'),
    ('Diego Alves', 'Curitiba', 'PR',
     'diego@email.com', '2026-05-05'),
    ('Elisa Rocha', 'Belo Horizonte', 'MG',
     'elisa@email.com', '2026-06-12');
```

Código 12: Inserção de cinco produtos

```
INSERT INTO produtos
    (nome, categoria, preco, estoque, ativo)
VALUES
    ('Notebook Pro 15', 'Informática', 4500.00, 12, TRUE),
    ('Mouse sem fio', 'Informática', 120.00, 50, TRUE),
    ('Cadeira Office', 'Móveis', 980.00, 8, TRUE),
    ('Monitor 27', 'Informática', 1600.00, 15, TRUE),
    ('Luminária LED', 'Casa', 180.00, 30, TRUE);
```

Código 13: Inserção de cinco vendedores

```
INSERT INTO vendedores
    (nome, setor, comissao, data_admissao)
VALUES
    ('Felipe Costa', 'Corporativo', 5.00, '2022-03-10'),
    ('Gabriela Reis', 'Varejo', 4.50, '2023-07-18'),
    ('Henrique Melo', 'Varejo', 4.00, '2024-01-22'),
    ('Isabela Nunes', 'Corporativo', 5.50, '2021-09-01'),
    ('João Martins', 'On-line', 3.50, '2025-02-14');
```

Código 14: Inserção de cinco vendas

```sql
INSERT INTO vendas
    (cliente_id, produto_id, vendedor_id, data_venda,
     quantidade, preco_unitario, status_venda)
VALUES
    (1, 1, 1, '2026-08-01', 1, 4400.00, 'CONCLUIDA'),
    (2, 2, 2, '2026-08-03', 3, 115.00, 'CONCLUIDA'),
    (1, 4, 5, '2026-08-07', 2, 1550.00, 'CONCLUIDA'),
    (3, 3, 4, '2026-08-10', 1, 950.00, 'PENDENTE'),
    (5, 5, 2, '2026-08-15', 4, 175.00, 'CANCELADA');
```

Preço atual e preço histórico

`produtos.preco` contém o preço atual do catálogo. Já `vendas.preco_unitario` registra o preço praticado na data da venda. Guardar esse valor histórico é importante: uma alteração futura no catálogo não deve mudar o valor de uma venda já realizada.

Ao final dessa etapa você já deve:

1. Ter um usuário diferente do administrador, com acesso ao banco de dados lojaVirtual, com privilégios para criar tabelas, inserir dados e fazer consultas.
2. Ter o banco de dados lojaVirtual com as tabelas e os dados inseridos.
3. Entender a estrutura e os dados das tabelas e as relações entre elas.
4. Estar em condições de fazer consultas nas tabelas.

8 Revisão dos comandos fundamentais

Antes de fazer consultas pedidas como exercícios, vamos rever alguns conceitos básicos. Estude os exemplos a seguir, entendendo o que fazem para depois poder fazer as consultas pedidas sem precisar consultar.

8.1 SELECT, aliases e expressões calculadas

Selecione dados da tabela vendas, mostrando o id da venda, a quantidade vendida, o preço do produto e o valor total da venda. Esse último deve ter o alias (apelido) 'valor_total'.

Código 15: Projeção de colunas e cálculo do total

```sql
SELECT
    venda_id,
    quantidade,
    preco_unitario,
    quantidade * preco_unitario AS valor_total
FROM vendas;
```

Um *alias*, criado com `AS`, fornece um nome temporário e mais claro a uma coluna ou tabela no resultado.

8.2 Filtros com WHERE

1) Faça uma consulta na tabela produtos mostrando produtos com preços entre 150 e 1700 e que estejam ativos.

2) Faça uma consulta na tabela clientes para mostrar o nome e a cidade de todos clientes de MG e SP e que tenham nomes que começam com a letra A.

Código 16: Exemplos de filtros

```sql
SELECT *
FROM produtos
WHERE preco BETWEEN 150.00 AND 1700.00
  AND ativo = TRUE;

SELECT nome, cidade
FROM clientes
WHERE uf IN ('MG', 'SP')
  AND nome LIKE 'A%';
```

Operadores importantes: `=`, `<>`, `>`, `>=`, `<`, `<=`, `BETWEEN`, `IN`, `LIKE`, `IS NULL`, `AND`, `OR` e `NOT`.

8.3 Ordenação e limitação

Faça uma consulta na tabela `produtos` mostrando o nome, a categoria e o preço de cada produto. Ordene os resultados primeiro pela categoria, em ordem alfabética crescente, e depois pelo preço, do maior para o menor dentro de cada categoria. Mostre somente os três primeiros registros do resultado.

Código 17: Ordenação por dois critérios

```sql
SELECT nome, categoria, preco
FROM produtos
ORDER BY categoria ASC, preco DESC
LIMIT 3;
```

8.4 Funções de agregação e agrupamentos

1) Faça uma consulta na tabela `produtos` que mostre, em uma única linha: a quantidade de produtos cadastrados, o menor preço, o maior preço, a quantidade total de unidades em estoque e o preço médio dos produtos. Arredonde o preço médio para duas casas decimais.

Código 18: Resumo estatístico dos produtos

```sql
SELECT
    COUNT(*) AS quantidade_produtos,
    MIN(preco) AS menor_preco,
    MAX(preco) AS maior_preco,
    SUM(estoque) AS estoque_total,
    ROUND(AVG(preco), 2) AS preco_medio
FROM produtos;
```

2) Agrupe os produtos por categoria e mostre, para cada categoria, a quantidade de produtos e o preço médio. Exiba somente as categorias cujo preço médio seja maior que R$ 500,00 e ordene o resultado do maior para o menor preço médio.

Código 19: Agrupamento com filtro de grupos

```sql
SELECT
    categoria,
    COUNT(*) AS quantidade,
    ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > 500
ORDER BY preco_medio DESC;
```

WHERE versus HAVING

`WHERE` filtra linhas antes do agrupamento. `HAVING` filtra os grupos produzidos por `GROUP BY`. Assim, uma condição como `preco > 100` costuma pertencer ao `WHERE`; uma condição como `AVG(preco) > 500` pertence ao `HAVING`.

8.5 UPDATE e DELETE

1) Escreva um comando que aumente em 5% o preço de todos os produtos pertencentes à categoria Informática.

2) Escreva um comando que exclua da tabela `vendas` todos os registros cujo status seja `CANCELADA`.

Código 20: Atualização e exclusão controladas

```sql
UPDATE produtos
SET preco = preco * 1.05
WHERE categoria = 'Informática';

DELETE FROM vendas
WHERE status_venda = 'CANCELADA';
```

Atenção

Antes de executar `UPDATE` ou `DELETE`, faça um `SELECT` com o mesmo `WHERE`. Sem o filtro, todas as linhas podem ser alteradas ou excluídas. Não execute os dois comandos anteriores antes das atividades, pois eles mudariam os dados de referência do roteiro.

9 Exercício 1: construção do banco

Prática de definição e manipulação de dados

1. Entre no MySQL como administrador e crie o usuário da aula.
2. Conceda a ele os privilégios necessários para criar e manipular o banco.
3. Conecte-se com o novo usuário e crie o banco `lojaVirtual`.
4. Crie as quatro tabelas, suas chaves e suas restrições.
5. Insira exatamente os cinco registros apresentados em cada tabela.
6. Use `SHOW TABLES`, `DESCRIBE` e `SHOW CREATE TABLE` para conferir o resultado.
7. Tente inserir uma venda com `cliente_id = 99`. Anote e explique o erro. Depois descarte esse comando inválido.

10 Exercício 2: consultas básicas e intermediárias

Escreva uma consulta para cada solicitação. São propostas 15 consultas, permitindo revisar mais conteúdos do que o mínimo solicitado.

Consultas 1 a 15

1. Liste todas as colunas e linhas da tabela `clientes`.
2. Mostre apenas nome, categoria e preço dos produtos.
3. Liste os clientes de Minas Gerais, em ordem alfabética.
4. Exiba os produtos cujo preço esteja entre R$ 150,00 e R$ 1.700,00.
5. Mostre os produtos da categoria Informática com estoque maior que 10.
6. Liste as três mercadorias mais caras.
7. Mostre as cidades distintas cadastradas em `clientes`.
8. Calcule o menor preço, o maior preço e o preço médio dos produtos.
9. Calcule a quantidade total de unidades mantidas em estoque.
10. Conte quantos clientes existem em cada UF e ordene do maior para o menor total.
11. Calcule a quantidade de produtos e o preço médio de cada categoria.
12. Mostre somente as categorias cujo preço médio seja superior a R$ 500,00.
13. Calcule, para cada venda, o valor total (`quantidade * preco_unitario`).
14. Calcule a soma das vendas concluídas, ignorando as pendentes e canceladas.
15. Conte as vendas por status e ordene do status mais frequente para o menos frequente.

11 Revisão de JOIN

Uma junção combina linhas relacionadas. A condição é escrita normalmente em `ON`, comparando a chave estrangeira com a chave primária correspondente.

Código 21: Venda com o nome do cliente e do produto

```sql
SELECT
    v.venda_id,
    c.nome AS cliente,
    p.nome AS produto,
    v.quantidade,
    v.preco_unitario,
    v.quantidade * v.preco_unitario AS valor_total
FROM vendas AS v
INNER JOIN clientes AS c
    ON c.cliente_id = v.cliente_id
INNER JOIN produtos AS p
    ON p.produto_id = v.produto_id;
```

INNER JOIN e LEFT JOIN

`INNER JOIN` mantém apenas as linhas que possuem correspondência nas duas tabelas. `LEFT JOIN` mantém todas as linhas da tabela à esquerda, mesmo quando não há correspondência à direita; nesse caso, as colunas da direita recebem `NULL`.

12 Exercício 3: consultas com JOIN

Junções 1 a 8

1. Liste cada venda com data, nome do cliente e nome do produto.
2. Acrescente à consulta anterior o vendedor, a quantidade e o valor total.
3. Mostre somente as vendas concluídas, com cliente, produto e valor total, em ordem decrescente de valor.
4. Calcule o total vendido por vendedor. Considere somente vendas concluídas.
5. Calcule o total comprado por cliente, incluindo com `LEFT JOIN` os clientes que não fizeram nenhuma compra. Use `COALESCE` para mostrar zero em vez de `NULL`.
6. Liste todos os produtos e a quantidade total de unidades vendidas. Inclua produtos sem vendas.
7. Mostre o total faturado por categoria, considerando vendas concluídas.
8. Liste os vendedores sem nenhuma venda, usando `LEFT JOIN` e `IS NULL`.

13 Gabarito comentado

13.1 Consultas básicas e intermediárias

Código 22: Soluções das consultas 1 a 5

```sql
-- 1
SELECT * FROM clientes;

-- 2
SELECT nome, categoria, preco FROM produtos;

-- 3
SELECT nome, cidade, uf
FROM clientes
WHERE uf = 'MG'
ORDER BY nome ASC;

-- 4
SELECT nome, preco
```

```sql
FROM produtos
WHERE preco BETWEEN 150.00 AND 1700.00;

-- 5
SELECT nome, estoque
FROM produtos
WHERE categoria = 'Informática' AND estoque > 10;
```

Código 23: Soluções das consultas 6 a 10

```sql
-- 6
SELECT nome, preco
FROM produtos
ORDER BY preco DESC
LIMIT 3;

-- 7
SELECT DISTINCT cidade
FROM clientes
ORDER BY cidade;

-- 8
SELECT MIN(preco) AS menor,
       MAX(preco) AS maior,
       ROUND(AVG(preco), 2) AS media
FROM produtos;

-- 9
SELECT SUM(estoque) AS unidades_em_estoque
FROM produtos;

-- 10
SELECT uf, COUNT(*) AS total_clientes
FROM clientes
GROUP BY uf
ORDER BY total_clientes DESC, uf ASC;
```

Código 24: Soluções das consultas 11 a 15

```sql
-- 11
SELECT categoria,
       COUNT(*) AS total_produtos,
       ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
GROUP BY categoria;

-- 12
SELECT categoria, ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > 500;

-- 13
SELECT venda_id,
```

```sql
      quantidade * preco_unitario AS valor_total
FROM vendas;

-- 14
SELECT SUM(quantidade * preco_unitario) AS faturamento
FROM vendas
WHERE status_venda = 'CONCLUIDA';

-- 15
SELECT status_venda, COUNT(*) AS total
FROM vendas
GROUP BY status_venda
ORDER BY total DESC, status_venda ASC;
```

13.2 Consultas com JOIN

Código 25: Soluções das junções 1 a 3

```sql
-- 1
SELECT v.venda_id, v.data_venda,
       c.nome AS cliente, p.nome AS produto
FROM vendas AS v
JOIN clientes AS c ON c.cliente_id = v.cliente_id
JOIN produtos AS p ON p.produto_id = v.produto_id;

-- 2
SELECT v.venda_id, v.data_venda,
       c.nome AS cliente, p.nome AS produto,
       vd.nome AS vendedor, v.quantidade,
       v.quantidade * v.preco_unitario AS valor_total
FROM vendas AS v
JOIN clientes AS c ON c.cliente_id = v.cliente_id
JOIN produtos AS p ON p.produto_id = v.produto_id
JOIN vendedores AS vd ON vd.vendedor_id = v.vendedor_id;

-- 3
SELECT c.nome AS cliente, p.nome AS produto,
       v.quantidade * v.preco_unitario AS valor_total
FROM vendas AS v
JOIN clientes AS c ON c.cliente_id = v.cliente_id
JOIN produtos AS p ON p.produto_id = v.produto_id
WHERE v.status_venda = 'CONCLUIDA'
ORDER BY valor_total DESC;
```

Código 26: Soluções das junções 4 a 6

```sql
-- 4
SELECT vd.nome AS vendedor,
       SUM(v.quantidade * v.preco_unitario) AS total_vendido
FROM vendedores AS vd
JOIN vendas AS v ON v.vendedor_id = vd.vendedor_id
WHERE v.status_venda = 'CONCLUIDA'
```

```sql
GROUP BY vd.vendedor_id, vd.nome
ORDER BY total_vendido DESC;

-- 5
SELECT c.nome AS cliente,
       COALESCE(SUM(CASE
         WHEN v.status_venda = 'CONCLUIDA'
         THEN v.quantidade * v.preco_unitario
         ELSE 0 END), 0) AS total_comprado
FROM clientes AS c
LEFT JOIN vendas AS v ON v.cliente_id = c.cliente_id
GROUP BY c.cliente_id, c.nome
ORDER BY total_comprado DESC;

-- 6
SELECT p.nome AS produto,
       COALESCE(SUM(CASE
         WHEN v.status_venda = 'CONCLUIDA'
         THEN v.quantidade ELSE 0 END), 0) AS unidades_vendidas
FROM produtos AS p
LEFT JOIN vendas AS v ON v.produto_id = p.produto_id
GROUP BY p.produto_id, p.nome
ORDER BY unidades_vendidas DESC;
```

Código 27: Soluções das junções 7 e 8

```sql
-- 7
SELECT p.categoria,
       SUM(v.quantidade * v.preco_unitario) AS faturamento
FROM vendas AS v
JOIN produtos AS p ON p.produto_id = v.produto_id
WHERE v.status_venda = 'CONCLUIDA'
GROUP BY p.categoria
ORDER BY faturamento DESC;

-- 8
SELECT vd.vendedor_id, vd.nome
FROM vendedores AS vd
LEFT JOIN vendas AS v ON v.vendedor_id = vd.vendedor_id
WHERE v.venda_id IS NULL;
```

14 Perguntas para discussão

Consolidação conceitual

Responda por escrito:

1. Qual é a diferença entre `PRIMARY KEY` e `FOREIGN KEY`?
2. Por que as tabelas independentes foram criadas antes de `vendas`?

3. Qual é a diferença entre `WHERE` e `HAVING`?
4. Em que situação `LEFT JOIN` produz um resultado diferente de `INNER JOIN`?
5. Por que o preço unitário também foi armazenado em `vendas`?
6. Por que um índice pode acelerar uma leitura e, ao mesmo tempo, aumentar o custo de inserções e atualizações?
7. Por que o MySQL pode ignorar um índice quando a tabela possui apenas cinco linhas?
8. Como o modelo deveria mudar para permitir vários produtos na mesma venda?

15 Checklist de aprendizagem

Resultado esperado

Ao final da revisão, verifique se você consegue:

- ◯ conectar-se ao MySQL com diferentes usuários;
- ◯ criar e inspecionar bancos e tabelas;
- ◯ explicar chaves primárias, estrangeiras e integridade referencial;
- ◯ inserir dados respeitando a ordem das dependências;
- ◯ filtrar, ordenar e agrupar linhas;
- ◯ aplicar funções de agregação corretamente;
- ◯ escolher entre `INNER JOIN` e `LEFT JOIN`;
- ◯ escrever uma subconsulta e criar uma view;
- ◯ explicar a função básica de um índice e do `EXPLAIN`;
- ◯ modificar ou excluir dados com segurança.